Step 4 — A Real Client That Connects and Uses All Three — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 4: a server nobody has connected to isn't proven to work. This step builds a standalone `client.py` — a real MCP client that launches the server, discovers what it offers at runtime, and uses all three building blocks in one run. The server itself doesn't change; this proves it from the outside, the way Claude Desktop or your own agent would. **If not:** you'd only have the Inspector's manual clicks, never the exact code path a real client uses, proven end to end.

After this step the project is complete: `python client.py` runs the full connect → discover → use loop in one go. `server.py` is unchanged from Step 3.

Solution — discover, then use all three, with a clear failure path

Story — `client.py`: `StdioServerParameters` launches `server.py` as a subprocess; `stdio_client` + `ClientSession` open the connection. The client calls `list_tools()` / `list_resources()` / `list_prompts()` to see what's on offer — "discovered, not hardcoded" (Doc06): it never imports anything from `server.py`. Then it uses each one by name. The whole thing is wrapped so a connection failure prints one clear line instead of a traceback, and the empty-note call shows the clean, readable rejection the server returns. **If not:** a missing `server.py` would dump a raw async traceback, and you'd never confirm the error path works from a client's side.

```
# client.py
# A standalone MCP client: launches server.py over stdio, discovers what it
# offers (list_tools/resources/prompts), then uses all three building blocks
# in one run. Wraps the connection so a failure prints one clear line.
import asyncio

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def main():
    server_params = StdioServerParameters(command="python", args=["server.py"])
    try:
        async with stdio_client(server_params) as (read, write):
            async with ClientSession(read, write) as session:
                await session.initialize()
```

```
                tools = await session.list_tools()
                tool_names = []
                for t in tools.tools:
                    tool_names.append(t.name)
                print("Tools:", tool_names)
                resources = await session.list_resources()
                resource_uris = []
                for r in resources.resources:
                    resource_uris.append(str(r.uri))
                print("Resources:", resource_uris)
                prompts = await session.list_prompts()
                prompt_names = []
                for p in prompts.prompts:
                    prompt_names.append(p.name)
                print("Prompts:", prompt_names)

                added = await session.call_tool(
                    "add_note", {"text": "Buy milk and eggs"}
                )
                print("add_note:", added.content[0].text)
                found = await session.call_tool(
                    "search_notes", {"query": "milk"}
                )
                print("search_notes:", found.content[0].text)
                res = await session.read_resource("notes://all")
                print("notes://all:", res.contents[0].text)
                prompt = await session.get_prompt(
                    "summarize_notes",
                    {"start_date": "2026-09-01", "end_date": "2026-09-30"},
                )
                print("summarize_notes:", prompt.messages[0].content.text[:60])

                bad = await session.call_tool("add_note", {"text": "   "})
                if bad.content[0].text.startswith("Error"):
                    print("empty note correctly rejected:",
                          bad.content[0].text)
                else:
                    print("WARNING: empty note was not rejected")
    except Exception as e:
        print(f"Could not start or connect to the notes server: {e}")

if __name__ == "__main__":
    asyncio.run(main())
```

Run it: `python client.py` (from the repo root). Every one of the four things the server offers is discovered and used in one run:

```
Tools: ['add_note', 'search_notes']
Resources: ['notes://all']
Prompts: ['summarize_notes']
add_note: Note added with id 1.
search_notes: Buy milk and eggs
notes://all: [1] Buy milk and eggs (2026-09-11T10:03:12.481022)
summarize_notes: Using the attached notes (see notes://all), summarize the
empty note correctly rejected: Error: note text cannot be empty or whitespace.
```

Prove the failure path: temporarily rename `server.py` and rerun `client.py` — instead of a raw async traceback, you get one clear line (`Could not start or connect to the notes server: ...`). Rename it back before you're done.

Note on `isError` vs `is_error`: with the current `mcp` SDK a `CallToolResult`'s flag is `result.is_error` (snake_case), not `isError`. This client checks the returned text instead, which works regardless: the server returns a clear `"Error: ..."` string for a rejected note (Step 2), so `bad.content[0].text.startswith("Error")` is the reliable, version-independent check.

Final Deliverable: MCPForge-Local-Notes-And-Tasks-Server — a real MCP server exposing 2 Tools (`add_note` , `search_notes`), 1 Resource (`notes://all`), and 1 Prompt (`summarize_notes`), backed by SQLite, plus a standalone `client.py` that proves the full connect → discover → use loop works end to end.

Why each is a Tool / Resource / Prompt (said plainly): `add_note` and `search_notes` *do* something — write, or run a query — so they're Tools. `notes://all` only hands back data to read, with nothing to act on — so it's a Resource. `summarize_notes` is a reusable, parameterized template, not a hardcoded string buried in one app — so it's a Prompt. Getting that split right, not the SQLite code, is what this project proves you can do.